



Intro to Dart

DMIT 2504 – WEEK 1 DAY 2



Announcements

Expectations - What I expect from you

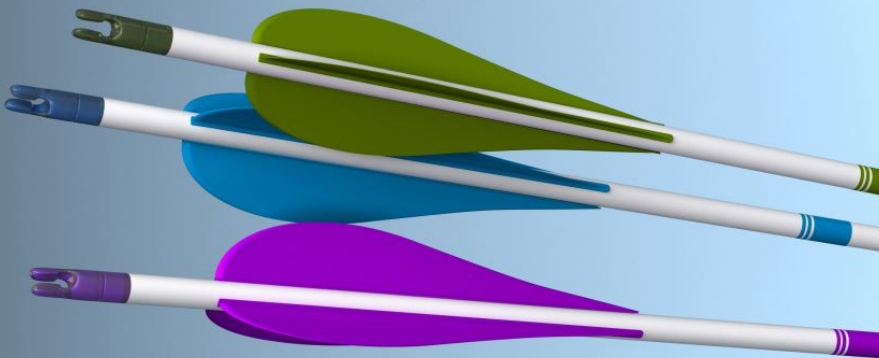
1. No Late Assignments
2. No Cheating
3. Be a good classmate
4. Don't go to stupid sites in class
5. Show up to class

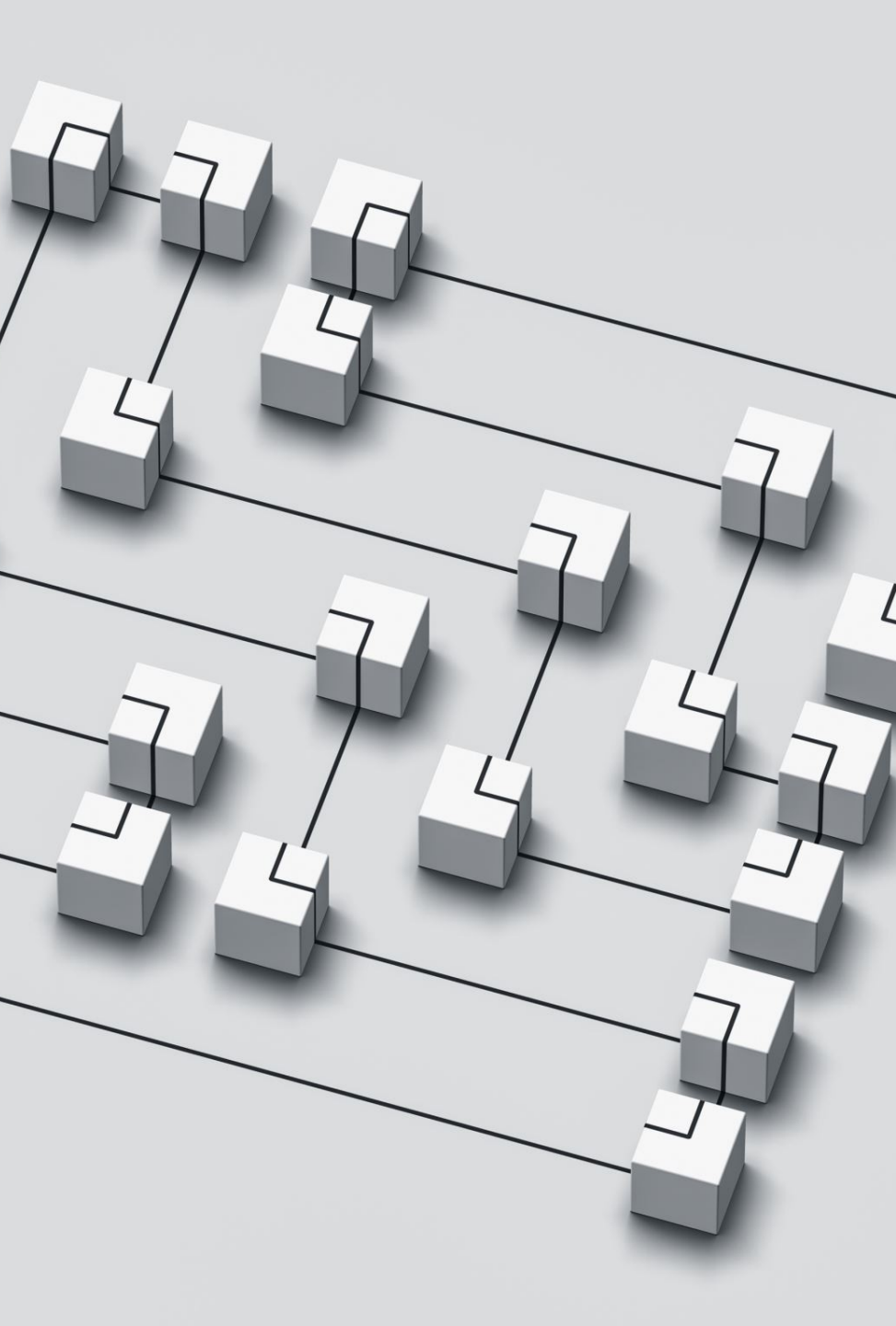
OBJECTIVES

After participating in this session, students should be able to ...

- Describe the Dart programming language
- Create a Dart console application project
- Run a Dart console application
- Capture input and display output in a Dart console application

The accompanying Dart files can be found and downloaded from the course LMS site

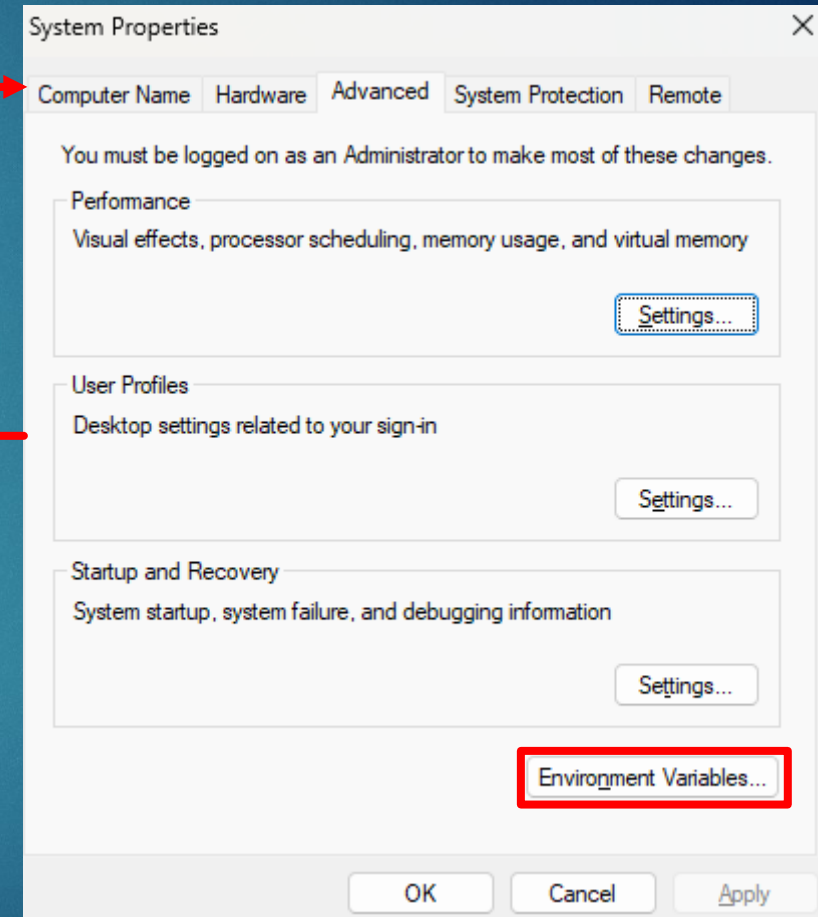
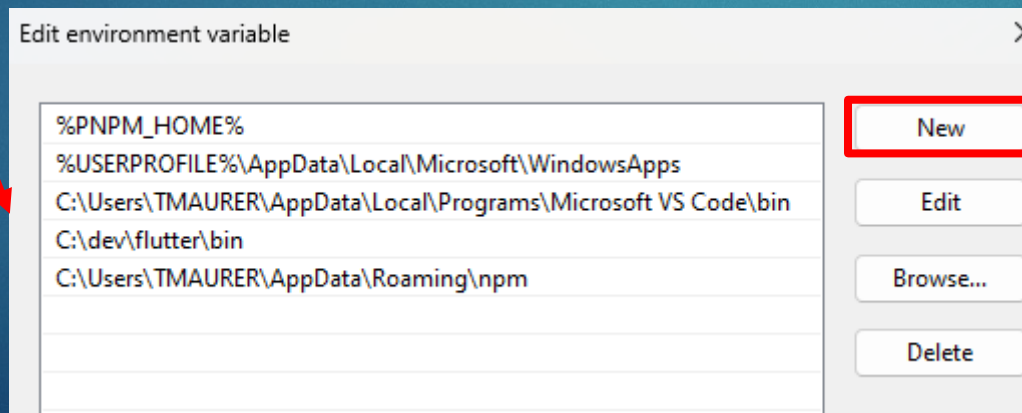
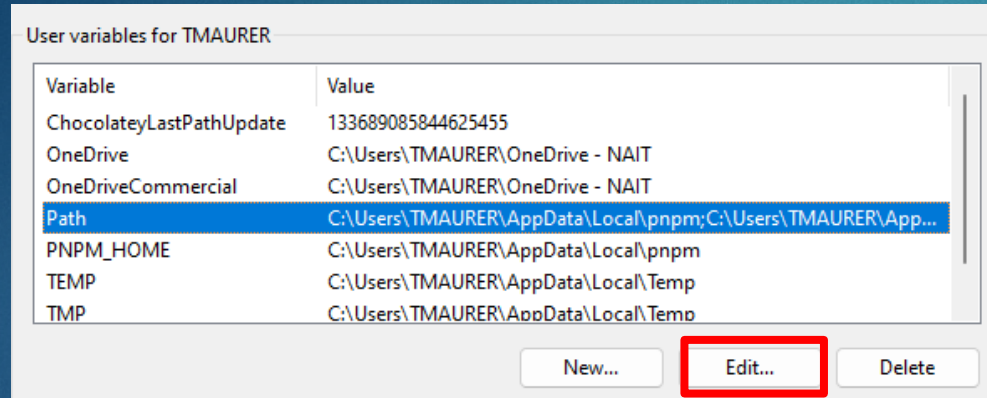
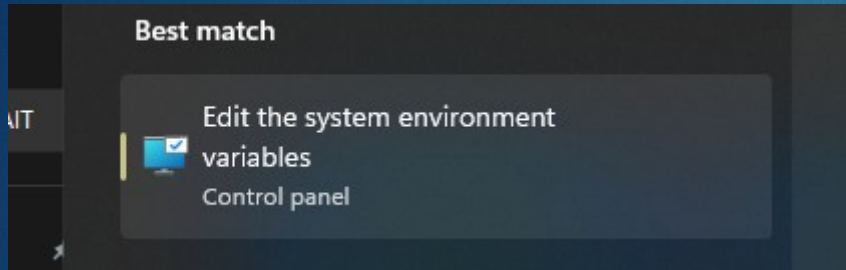




Question from last class: What is the PATH?

PATH is an environment variable on several operating systems Microsoft Windows, specifying a set of directories where executable programs are located.

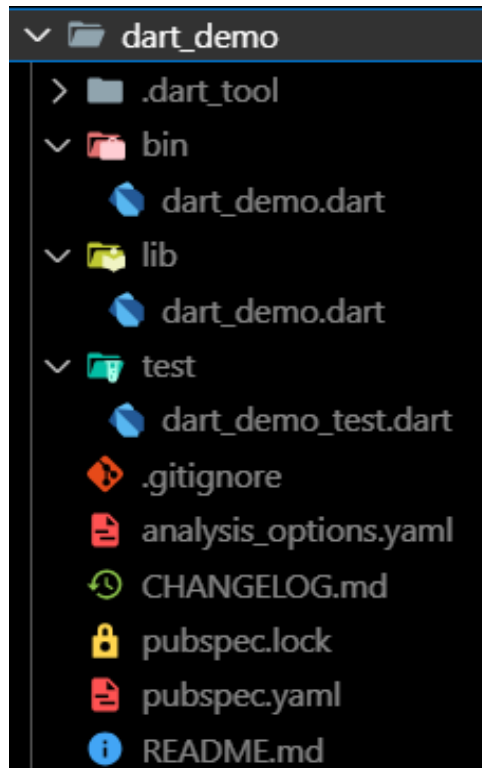
How to PATH?



DART LANGUAGE

- ▶ Dart, created by Google, is a client-optimized programming language for fast apps on any platform
 - ▶ [Dart Programming Language](#)
- ▶ Designed for building mobile, desktop, server, and web applications
- ▶ Ahead-of-time (AOT) compiled to native code for fast execution
- ▶ Learn quickly with DartPad, an online tool for running Dart code directly in the browser
 - ▶ [DartPad](#)
- ▶ Code style and conventions are documented online
 - ▶ [Effective Dart | Dart](#)

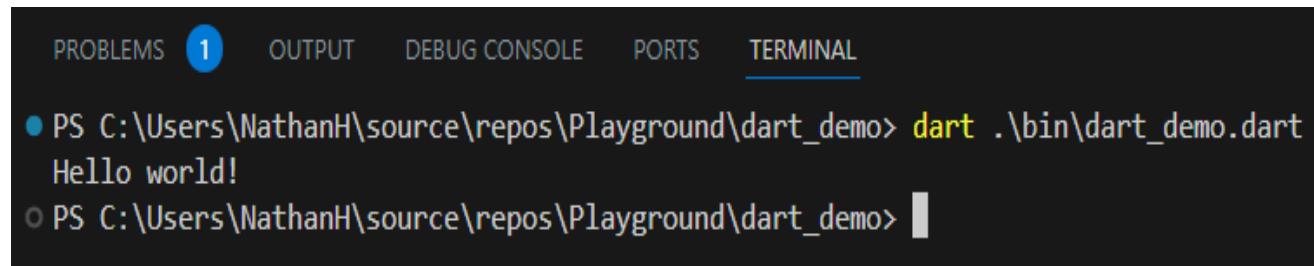
Dart Console Application



- ▶ Create a Dart Console Application in VS Code
 - ▶ Ctrl+Shift+P `Dart: NewProject` `Console Application`
 - ▶ Follow the appropriate naming convention
- ▶ Dart project directory structure:
 - ▶ bin -the main executable is contained here (main method)
 - ▶ lib - additional library code for your project can be placed here
 - ▶ test -unit tests can be placed here

RUNNING THE APPLICATION

- ▶ Open a terminal
- ▶ Navigate to the project directory (use the `cd` command)
- ▶ Execute your Dart program using the `dart` command
 - ▶ `dart.\bin\dart_demo.dart`

A screenshot of a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'PORTS', and 'TERMINAL'. The 'TERMINAL' tab is active and underlined. Below the tabs, there are two lines of text. The first line shows a command prompt 'PS C:\Users\NathanH\source\repos\Playground\dart_demo>' followed by the command 'dart .\bin\dart_demo.dart' in yellow, and the output 'Hello world!'. The second line shows the same command prompt followed by a cursor, indicating the terminal is ready for the next command.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE PORTS TERMINAL
PS C:\Users\NathanH\source\repos\Playground\dart_demo> dart .\bin\dart_demo.dart
Hello world!
PS C:\Users\NathanH\source\repos\Playground\dart_demo> 
```

DART CONSOLE OUTPUT

- ▶ You can use the `stdout` object from the `dart:io` library to display text on the console
 - ▶ `write()` will write text and leave the cursor on the same line
 - ▶ `writeln()` will write text and move the cursor to the next line
- ▶ While Dart allows both single and double quotes to be used to delimit strings, single quotes are preferred
- ▶ Interpolate variables using a `$` or `${}` for expressions or `.accessor`

```
dart_demo > bin > dart_demo.dart > ...  
1  import 'dart:io';  
2  
   Run | Debug  
3  void main(List<String> arguments) {  
4      stdout.writeln('Hello world!');  
5  }
```

DART CONSOLE INPUT

- ▶ You can use the `stdin` object from the `dart:io` library to read in a string from the console
 - ▶ `stdin.readLineSync()` will read in the next line as a string
- ▶ Types can be used to parse strings into other types
 - ▶ `int.parse()`
 - ▶ `double.parse()`

```
dart_demo > bin > dart_demo.dart > ...
1  import 'dart:io';
2
   Run | Debug
3  void main(List<String> arguments) {
4      stdout.write('Enter your name: ');
5      final name = stdin.readLineSync();
6      stdout.writeln('Hello $name, how are you?');
7  }
```

ADD LIBRARY FUNCTIONS

- ▶ You can create functions in a library file and import those functions into your program

```
dart_demo > lib > dart_demo.dart > ...
1  import 'dart:io';
2
3  /// Returns a [String]
4  ///
5  /// Displays the passed in [message] as the prompt for the user.
6  String prompt(String message) {
7    stdout.write(message);
8    String? input = stdin.readLineSync();
9    return input ?? '';
10 }
```

```
dart_demo > bin > dart_demo.dart > ...
1  import 'dart:io';
2
3  import 'package:dart_demo/dart_demo.dart';
4
5  Run | Debug
6  void main(List<String> arguments) {
7    final name = prompt('Enter your name: ');
8    stdout.writeln('Hello $name, how are you?');
9  }
```

SUMMARY

- ▶ Dart is a client-optimized programming language for targeting any platform
- ▶ VS Code can be used to quickly create Dart projects
- ▶ The `dart:io` library has classes for displaying output and capturing input from the console

```
/// Returns an [int]
///
/// Will continue to prompt the user until a valid
/// Displays the passed in [message] as the prompt
int promptInt(String message) {
    bool isValid = false;

    while (!isValid) {
        String input = prompt(message);
        try {
            return int.parse(input);
        } catch (e) {
            stdout.writeln('Invalid input. Please enter a
        }
    }
}
```